1 Projet : VNF Chain Optimization

Ce notebook formalise la logique et la modélisation mathématique pour résoudre le projet d'optimisation (placements de VNF sur des serveurs).

1.1 Objectif Global

Trouver un placement optimal de n fonctions réseau virtuelles (VNF) sur m serveurs physiques qui minimise la charge maximale parmi tous les serveurs (le goulot d'étranglement ou **cycle time** CT), tout en respectant les contraintes de précédence entre les VNF.

---

2 Phase 1 : Solution Exacte (MILP)

Cette phase formalise le problème comme un programme linéaire en nombres entiers (MILP) et utilise Gurobi pour trouver la solution optimale.

2.1 1. Formulation Mathématique

2.1.1 1.1 Variables de Décision

- $x_{jk} \in \{0, 1\}$: Vaut 1 si la VNF j est placée sur le serveur k, 0 sinon.
- $s_j \in \{1, \ldots, m\}$: Indice du serveur sur lequel la VNF j est placée.
- $CT \geq 0$: Le "Cycle Time" ou latence du goulot d'étranglement (charge maximale d'un serveur).

2.1.2 1.2 Fonction Objectif

Minimiser CT

2.1.3 1.3 Contraintes

1. **Unicité du placement** : Chaque VNF j doit être placée sur exactement un serveur.
$$\sum_{k=1}^{m} x_{jk} = 1 \quad \forall j \in J$$
2. **Définition de l'indice du serveur** : Lien entre les variables binaires x et l'indice s_j.
$$s_j = \sum_{k=1}^{m} k \cdot x_{jk} \quad \forall j \in J$$
3. **Respect des précédences** : Pour chaque arc $(i, j) \in P$, la VNF i doit être placée sur le même serveur ou un serveur précédent par rapport à j.
$$s_i \leq s_j \quad \forall (i, j) \in P$$

4. **Goulot d'étranglement** : La somme des temps de traitement sur n'importe quel serveur k ne doit pas dépasser le cycle time.

$$\sum_{j=1}^{n} t_j \cdot x_{jk} \leq CT \quad \forall k \in K$$

2.2 2. Résultats et Analyse

2.2.1 2.1 Tableau des résultats

Le tableau suivant regroupe les résultats de notre modèle MILP sur les 24 instances du benchmark SALBPGen. Le nombre de serveurs m est spécifique à chaque instance (issu du benchmark). Une limite de temps de 30 minutes par instance a été imposée.

Instance	$\boldsymbol{n}$	$\boldsymbol{m}$	**OS**	**Préc.**	$\sum t_j$	$\boldsymbol{CT^*}$	**Temps (s)**
n=20_1	20	3	0.268	16	2882	962	0.054
n=20_2	20	3	0.300	19	2861	954	0.007
n=20_3	20	3	0.300	18	2785	930	0.056
n=20_4	20	3	0.289	18	2727	909	0.036
n=20_5	20	3	0.289	18	2837	950	0.076
n=20_6	20	3	0.295	19	2914	973	0.045
n=20_7	20	3	0.300	18	2786	930	0.041
n=20_8	20	3	0.289	20	2985	996	0.071
n=20_9	20	3	0.289	18	2925	976	0.055
n=20_10	20	3	0.279	17	2831	945	0.067
n=20_11	20	3	0.279	18	2766	924	0.089
n=20_12	20	3	0.300	17	2930	977	0.023
n=20_13	20	3	0.268	17	2896	967	0.078
n=20_14	20	3	0.263	17	2979	994	0.086
n=20_15	20	3	0.295	18	2981	994	0.037
n=20_16	20	12	0.232	17	10376	981	0.751
n=20_17	20	10	0.295	17	9134	969	0.366
n=20_18	20	11	0.300	20	9524	960	0.191
n=20_19	20	14	0.268	18	11113	1000	0.240
n=20_20	20	11	0.295	17	9829	1000	0.298
n=50_1	50	8	0.196	58	7276	910	11.52
n=50_2	50	6	0.195	56	5585	931	1.49
n=100_1	100	23	0.196	105	22723	N/A†	>1800
n=100_2	100	21	0.195	104	20262	N/A†	>89

Table 1 – Résultats complets de l'optimisation MILP (Phase 1). †Instances n'ayant pas convergé dans la limite de 30 minutes.

2.2.2 2.2 Analyse graphique des performances

La figure 1 illustre l'évolution du temps de résolution en fonction de la taille de l'instance et de l'Order Strength.

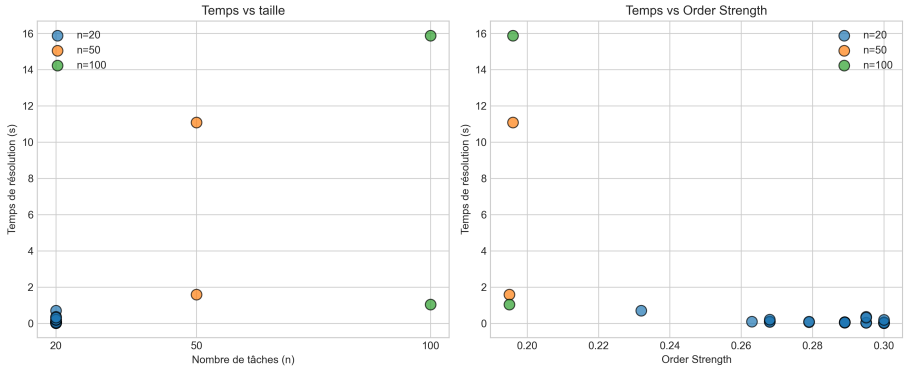

FIGURE 1 – Temps de résolution en fonction de la taille du problème (n) et de l'Order Strength (OS).

2.3 Conclusions

- **Instances** $n = 20$: Toutes les 20 instances sont résolues à l'optimalité en moins d'une seconde (≤ 0.75s). Les CT^* obtenus respectent la borne $CT \leq 1000$ imposée par le benchmark. Les instances 1–15 ($m = 3$) sont les plus rapides ; les instances 16–20 ($m \geq 10$), plus coûteuses en nombre de variables binaires, restent néanmoins résolues très efficacement.
- **Instances** $n = 50$: Résolues à l'optimalité en 1.5s à 11.5s. L'augmentation significative du temps pour n=50_1 ($m = 8$) par rapport à n=50_2 ($m = 6$) illustre l'impact du nombre de serveurs sur la combinatoire ($n \times m$ variables binaires).
- **Instances** $n = 100$: Non résolues dans la limite de 30 minutes. Avec $m \geq 21$ serveurs, le modèle comporte plus de $n \times m = 2100$ variables binaires, rendant l'exploration du Branch-and-Bound très coûteuse. Cela motive le recours aux heuristiques (Phase 2).
- **Impact de** m **vs.** n : Le facteur déterminant pour la difficulté n'est pas uniquement n mais le produit $n \times m$, qui détermine le nombre de variables binaires x_{jk}.